Föreläsning 7

Testdriven utveckling

Grundidé

1. Skriv ett test innan funktionaliteten är implementerad
2. Kör testet — det ska misslyckas
3. Skriv **minsta möjliga** kod för att få testet att lyckas
   - Håll fokus på kodens funktionalitet
   - Lägg alltså inte ner extratid på kodkvalitet, läsbarhet…, det kan göra att man tappar fokus
4. Refaktorisera koden
   - jobba med kodkvalitet, läsbarhet, flexibilitet, reducera koduppprepningar
5. Upprepa ovanstående

Detta kallas ofta **Red** → **Green** → **Refactor**.

Varför använda TDD?

- Bättre kodkvalitet
- Enklare underhåll
- Färre buggar
- Upptäcka ev. oduglig vald datastruktur i god tid
- Förbättrad design genom kortare, tydligare metoder
- Trygghet vid refaktorering
- Dokumentation genom tester

Vanliga utmaningar

- Upplevs långsamt i början
- Kräver disciplin
- Svårt utan bra kravspecifikation
- Ovana vid att tänka i testscenarier

Utveckling av Shut the Box med TDD

Shut the Box är ett tärningsspel med brickor som:



Kan spelas av en eller flera personer

9 brickor numrerad från 1-9

2 tärningar

Spelomgång:

1. Alla brickor är uppfällda
2. Spelaren kastar tärningarna
3. Summan av tärningarna avgör vilka brickor som kan fällas ner.
   T.ex om man får ögonen 6 och 1 på tärningarna så får man summan 7. Man kan då välja att fälla ner brickan 7 eller någon av kombinationerna som ger summan 7 d.v.s: [5,2], [3,4] ,[1,6] eller [1,2,4]
4. Spelet är slut när alla brickor är nedfällda eller inga kombinationer kan ge den summan
5. Poängen är summan av de brickor som fortfarande är uppfällda. Målet är att få 0 poäng.

Specifikation över olika delar av spelet

- Klassen ShutTheBox:
  - Datastruktur: brickorna har unika siffror 1-9 så ett Set<Integer> skulle kunna funka
  - Metoderna:
    - `boolean isTileUp(int tile)`
    - `boolean canPlay(int diceSum)`
    - `boolean takeTiles(int diceSum, int… tiles)`
    - `boolean isGameOver()`
    - `int getScore()`

Skapa Testfilen ShutTheBoxTest

```
import static org.junit.Assert.*;
import org.junit.Test;

public class ShutTheBoxTest {

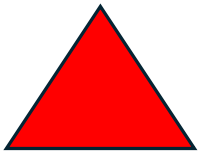
    @Test
    public void newGame_allTilesAreUp() {
        ShutTheBox game = new ShutTheBox();
    }
}
```

Steg1, Kompilerar `ShutTheBoxTest.java`

```
javac -cp lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar:. ShutTheBoxTest.java
```

Ger kompileringsfelet nedan:

```
ShutTheBoxTest.java:8: error: cannot find symbol
        ShutTheBox game = new ShutTheBox();
        ^
  symbol:   class ShutTheBox
  location: class ShutTheBoxTest
ShutTheBoxTest.java:8: error: cannot find symbol
        ShutTheBox game = new ShutTheBox();
                              ^
  symbol:   class ShutTheBox
  location: class ShutTheBoxTest
2 errors
```



Skapa filen `ShutTheBox.java`

```
//Filen ShutTheBox.java
public class ShutTheBox {
}
```

Kompilerar igen:

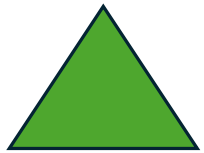
```
javac -cp lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar:.
ShutTheBoxTest.java
```

Inga kompileringsfel, vi kör testet:

```
java -cp lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar:.
org.junit.runner.JUnitCore ShutTheBoxTest
```

Vi får:

```
JUnit version 4.13.2.
Time: 0,005
OK (1 test)
```



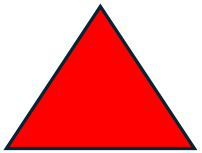
Steg1, Skriv Test1: Alla brickor ska skapas

- Vi lägger till mer textkod för att se om brickorna är skapade:

```
import static org.junit.Assert.*;
import org.junit.Test;

public class ShutTheBoxTest {

    @Test
    public void newGame_allTilesAreUp() {
        ShutTheBox game = new ShutTheBox();
        for (int i = 1; i <= 9; i++) {
            assertTrue(game.isTileUp(i));
        }
    }
}
```



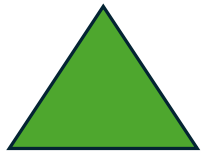
Implementerar så lite som möjligt tills testet passerar igenom

```
import java.util.*;

public class ShutTheBox {
    private Set<Integer> u;

    public ShutTheBox() {
        u = new HashSet<>();
        for (int i = 1; i <= 9; i++) u.add(i);
    }

    public boolean isTileUp(int i) {
        return u.contains(i);
    }
}
```



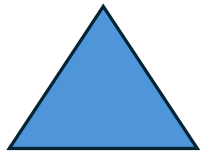
Refaktorisering, behövs ???

```
import java.util.*;

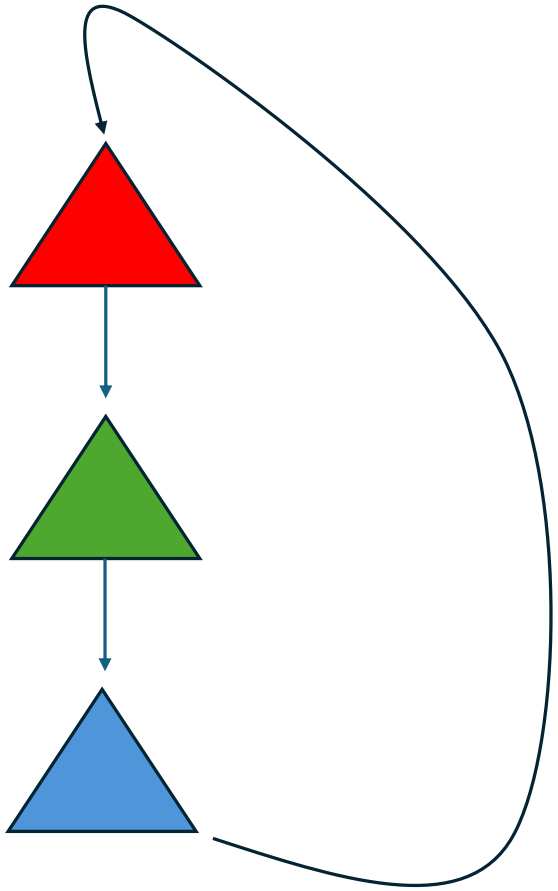
public class ShutTheBox {
    private Set<Integer> up;

    public ShutTheBox() {
        up = new HashSet<>();
        for (int i = 1; i <= 9; i++) up.add(i);
    }

    public boolean isTileUp(int tile) {
        return up.contains(tile);
    }
}
```



Också fortsätter....

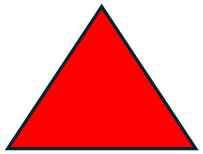


TDD + git/github

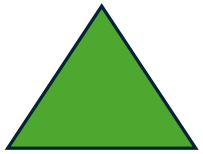
- Laboration 5 måste implementeras med TDD-metoden. Och du ska kunna redovisa att implementerad kod är ett resultat av TDD som programutvecklingsteknik.
- Kombinera detta TDD med git och github för att kunna redovisa alla steg från ingen kod alls till fungerande kod med hög kodkvalitet.
- När ska man committa och pusha?



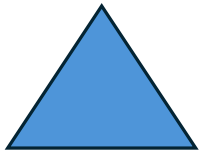
Exempel på Commit-meddelanden



RED: Lagt till test för metoden `isTileUp`



GREEN: Implementerat metodden `isTileUp`



REFACTOR: reducerat koduppprepning från `isTileUp`